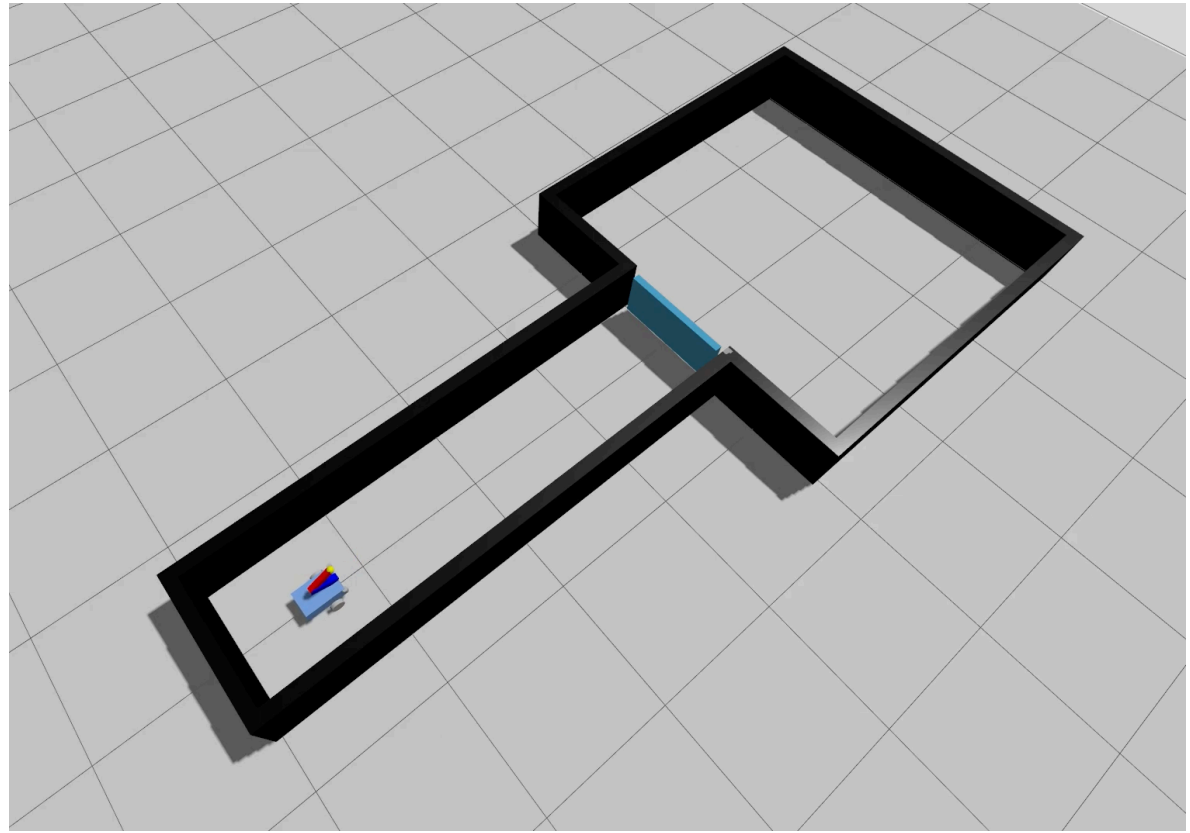
MiniLab 2 — Arm Kinematics & Mobile Manipulation

Autonomous Systems and Mobile Robots · Week 7



Janosch Bajorath
j.bajorath@uni-muenster.de

Universität Münster

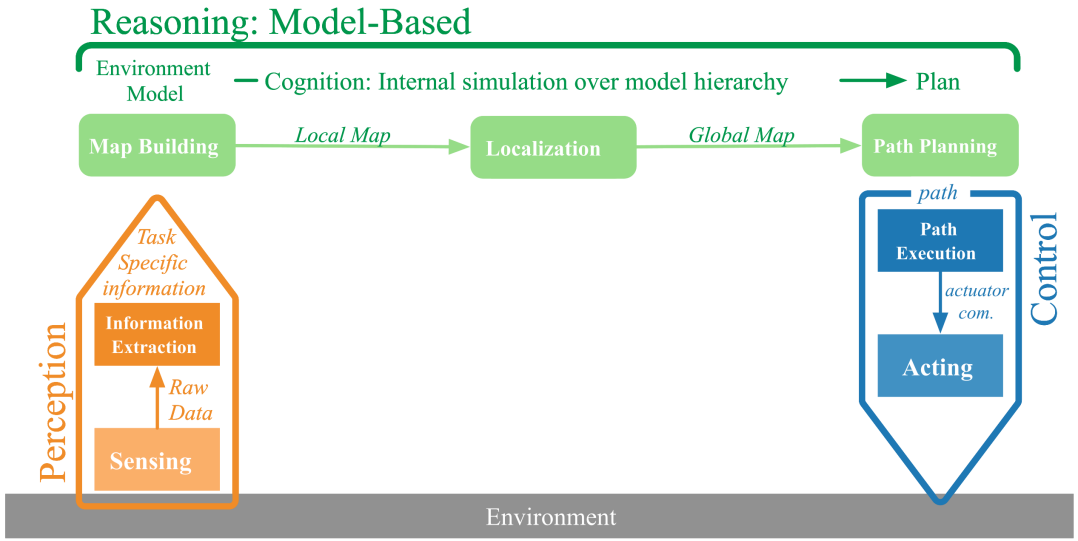
Recap

Where you are, where we go today

Semester at a Glance

#	Date	Theme
0	17.4	Python Intro + PRA Loop
1	24.4	ROS2 Basics + PID Control
—	*1.5*	*Public holiday — no session*
2	8.5	Robot Body and Blind Locomotion
3	15.5	Exteroception — LiDAR
4	22.5	MiniLab 1 Briefing
—	*29.5*	*Pfingsten — no session*
5	5.6	Troubleshooting — MiniLab 1
6	12.6	FK/IK Robotic Manipulator
7	**19.6**	**MiniLab 2 Briefing** ← you are here
8	26.6	Troubleshooting — MiniLab 2
9	3.7	Recap

Perceive · Reason · Act



Perceive

`/scan` — in Task 2 the LiDAR detects the door blocking the corridor exit.

Reason

FK/IK — where must the end-effector go to push it? The control stack (`kinematics_server` , `trajectory_server`) handles this.

Act

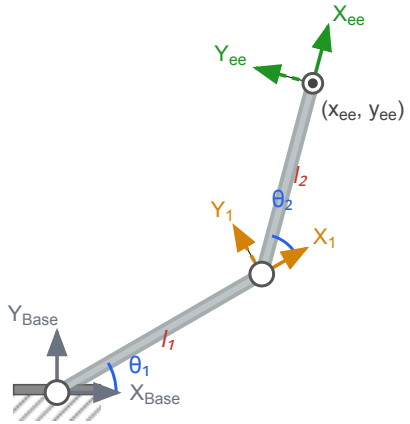
The `arm_pid_controller` closes the loop; the mission drives the stack via `execute_trajectory` .

Last Week

Week 6 — FK/IK Robotic Manipulator

- Homogeneous transforms — chain one per joint
- **Forward kinematics**: joint angles → end-effector pose
- **Geometric IK**: target → joint angles (elbow-up / elbow-down)
- The Jacobian and singularities

> You derived this arm's kinematics in the notebook. Now you put it into ROS2.



Two-link arm — the geometry you derived last week.

Today's Content

New Concepts

- Services vs. actions
- The action protocol
- Executors & callback groups
- Smooth joint motion
- Commanding a Gazebo arm

Task 1 — build the stack

- FK/IK services
- Tune the controller
- Trajectory action server
- `push_block_mission`
- *on a fixed pedestal arm*

Task 2 — run the mission

- Mount the arm on the mobile base
- LiDAR corridor nav
- Push open the door
- *stack is a black-box action*

Logistics

Gotchas, grading, getting started

Grading and Submission

Task	Item	Pts
1a	Kinematics services	5
1b	Tune the controller — holds a step	5
1c	Trajectory action — full protocol, smooth	10
1d	`push_block_mission` — block displaced	5
2a	Mount the arm	5
2b	Single-command bring-up	5
2c	Drive the corridor (LiDAR)	5
2d	Push open the door · enter the room	10
—	**Total (pass at 25)**	**50**

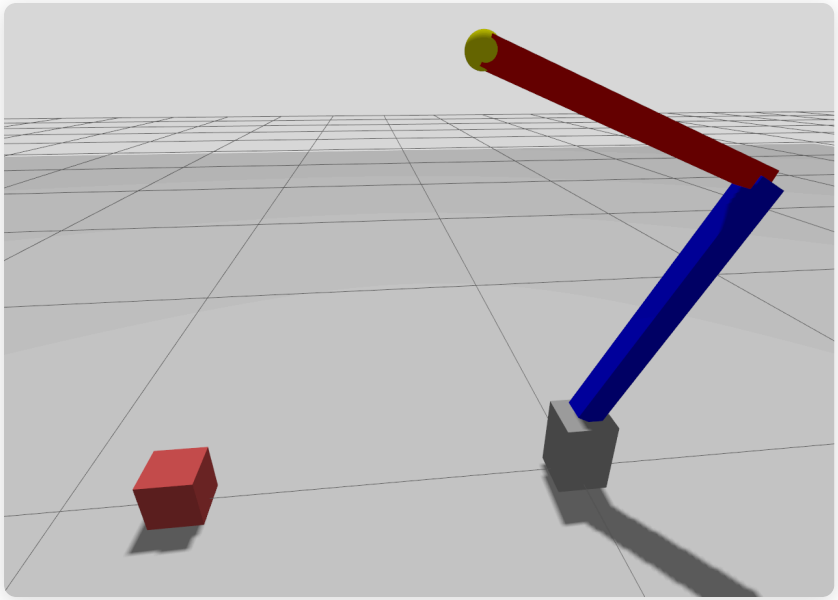
Submit	one ZIP of the ROS2 workspace
Build	from scratch with `colcon`
Omit	`build/ install/ log/`
Name	`minilab2_lastname1_lastname2.zip`
Issued	19.06.2026
Due	01.07.2026, 23:59

Questions: j.bajorath@uni-muenster.de

The Arm Control Stack

Build the reusable control stack on a fixed pedestal arm

The Arm



The two-link arm in Gazebo — blue shoulder (l1), red elbow (l2), yellow End Effector.

GEOMETRY

Same as Week 6 — two revolute joints (shoulder + elbow), moves in a **single plane**.

Link lengths come from a YAML file — not hardcoded:

`Task 1 (pedestal)` $l_1 = l_2 =$ **0.50 m**

`Task 2 (mobile)` $l_1 = l_2 =$ **0.20 m**

TASK 1 — BUILD THE ARM CONTROL STACK

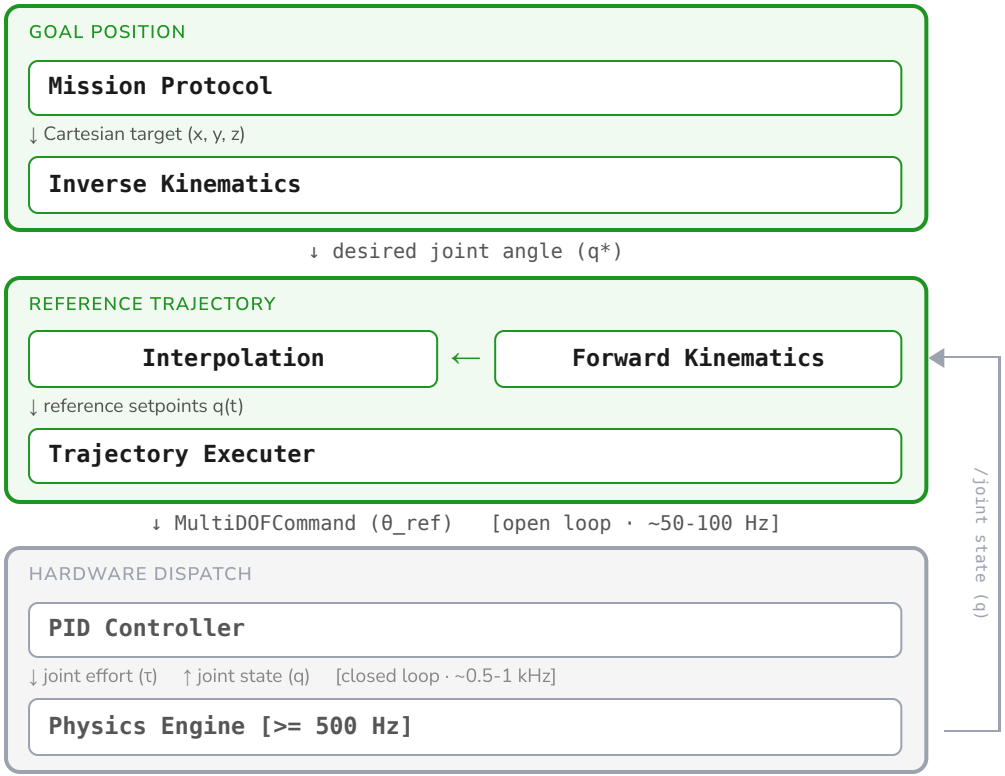
(a) FK / IK services — port your Week 6 kinematics into two ROS 2 services

(b) Tune the controller — set the blank PID gains so the arm holds a commanded pose

(c) Trajectory action server — move through a list of Cartesian waypoints, smoothly

(d) Push mission — use the action to drive the arm through home → approach → push

From Goal to Physics



Reference generation — `trajectory_server` interpolates q* into a smooth setpoint stream and publishes it to `/arm_pid_controller/reference`.

Closed-loop tracking — `arm_pid_controller` reads `/joint_states` and drives effort to track the reference. This is the Week-1 PID on a real manipulator — "where is the controller?" answered.

ROS2 Joint Control Stack

Joint Control Interfaces: setting reference angles and reading joint state via ros2_control.

Gazebo simulates physics — joints need a *force*, not an angle command.

- `pid_controller/PidController` converts your reference angle into torque by comparing it to the current joint position
- `ros2_control` hosts the controller; `gz_ros2_control` bridges it to the Gazebo physics engine

Integrating pid_controller/PidController via ros2_control and gz_ros2_control

Command interface — publish `MultiDOFCommand` (θ_ref) to `/arm_pid_controller/reference`

State interface — read `/joint_states` ; index by **joint name**, not array position

Joint Trajectories for Smooth Motion

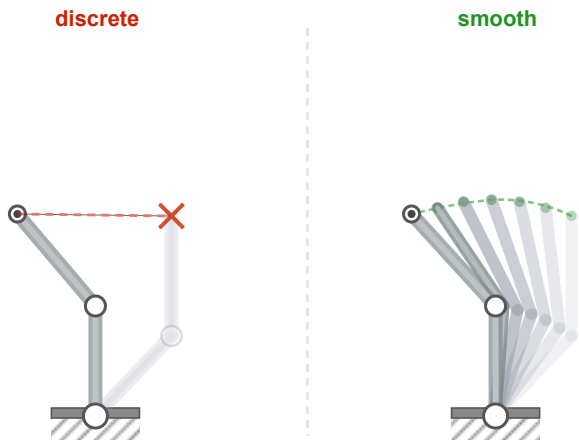
Discrete - One command per waypoint

Send q* once and the PID controller drives toward it as fast as it can — no pacing. The arm **snaps and lurches** between waypoints.

Smooth - A stream of reference setpoints

`trajectory_server` interpolates from the current pose toward the next waypoint and streams small steps at ~50 Hz — the PID tracks each one and the joints **sweep smoothly**.

F



Step count and publish rate are yours to choose in `trajectory_server`.

Recap - The Action Protocol

A service answers once. Driving an arm through a trajectory **takes time** — so it is an **action**, with a four-part contract.

Goal
the waypoints to execute — accepted or rejected

Feedback
progress, streamed *while* the arm moves

Result
the outcome + final configuration, on completion

Cancel
a request to stop the arm mid-motion

You implement **both halves**: `trajectory_server` is the action **server**; `push_block_mission` is the **client** that sends the goal and follows it through.

How Servers Stay Responsive

An action server must handle feedback, cancels, and joint-state updates *while* the goal callback is running — that depends on the **executor**, ROS2's callback dispatcher.

SingleThreadedExecutor (default)

```
# execute_cb — trajectory loop
for wp in goal.request.waypoints:
    self.move_to(wp)
    time.sleep(1.0)   # thread frozen here
    goal_handle.publish_feedback(fb)
```

While sleeping, no other callback runs — cancels ignored, feedback stalled, node hangs silently.

MultiThreadedExecutor + ReentrantCallbackGroup

```
# Same callback — nothing changes here
for wp in goal.request.waypoints:
    self.move_to(wp)
    time.sleep(1.0)
    goal_handle.publish_feedback(fb)
```

```
# Fix lives in node setup
cb = ReentrantCallbackGroup()
ActionServer( ... , callback_group=cb)
MultiThreadedExecutor().add_node(self)
```

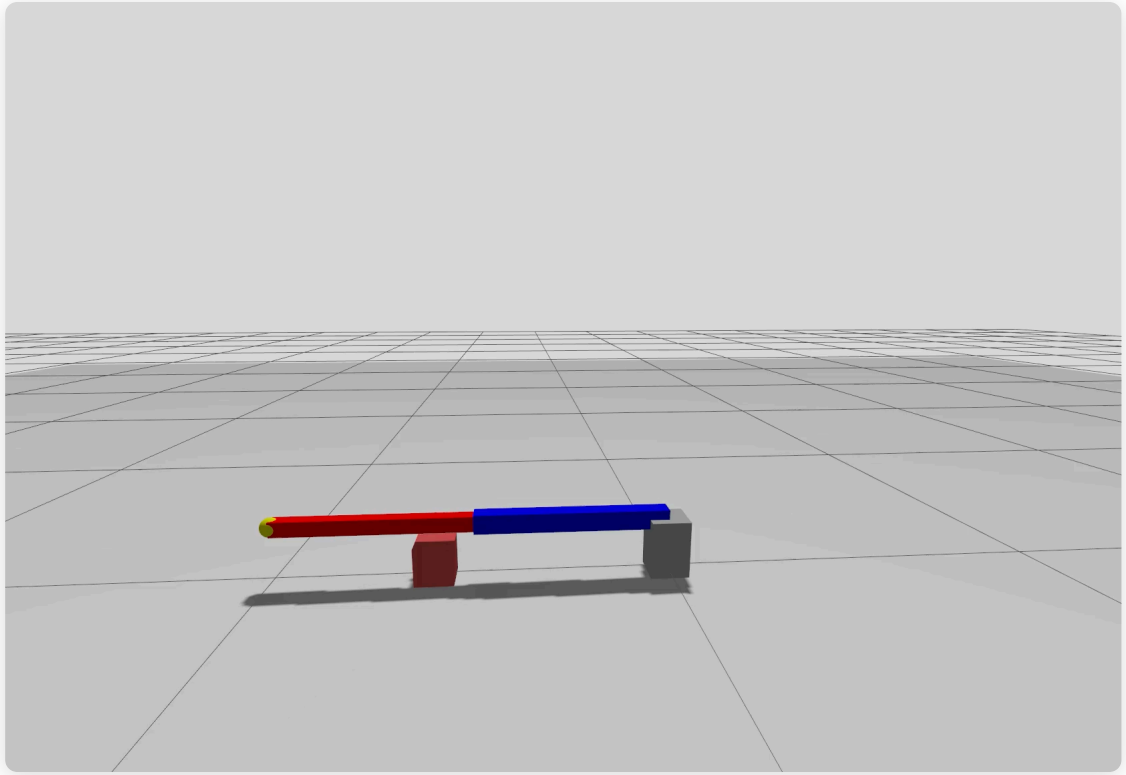
MultiThreadedExecutor — thread pool; callbacks can run in parallel.
ReentrantCallbackGroup — permission for callbacks in this group to overlap. Without it, the executor still serialises them.

Starting Point

Package	Role	Origin
`asmr_arm_description`	fixed-arm URDF + Gazebo, push world, controller config (template)	provided
`asmr_arm_bringup`	**you create the package:** `push_block_world.launch.py` + `ros_gz` **bridge config**	you build
`asmr_arm_interfaces`	`ComputeFK.srv`, `ComputeIK.srv`, `ExecuteTrajectory.action` — pinned contract	provided
`asmr_arm_control`	`kinematics_server`, `trajectory_server`	you build
`asmr_arm_mission`	`push_block_mission` — **action client, sends waypoints**	you build

Launch: `ros2 launch asmr_arm_bringup push_block_world.launch.py` *· Run:* `ros2 run asmr_arm_mission push_block_mission`

Solution Preview

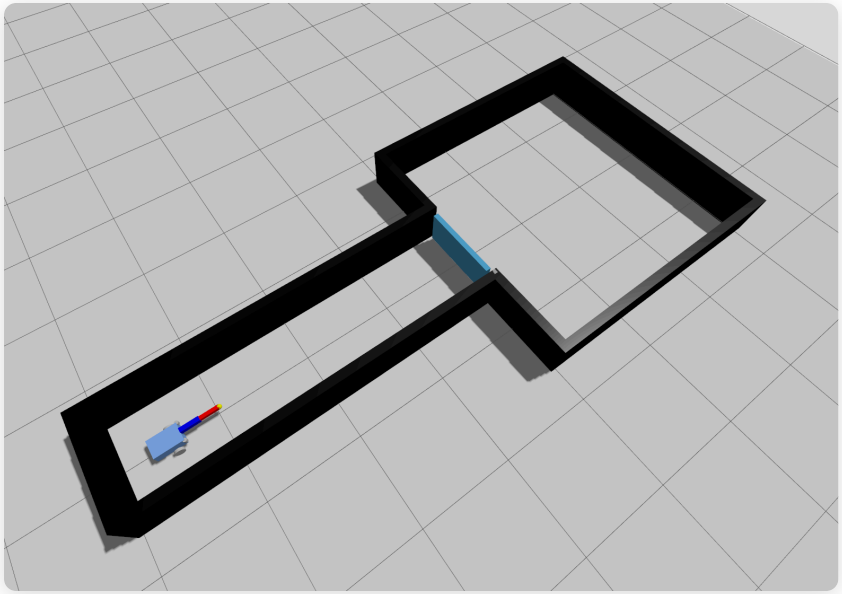


Three named waypoints displace the block.

The Mobile Manipulation Mission

The same arm, on your robot, in a corridor

The Mission



The straight-corridor world — hinged door at the exit.

1. Drive the **corridor** using **LiDAR**
2. Detect the **door** blocking the exit → **push it open with the arm**
3. **Pass through** once the way is clear

Validity rule. Navigation and arm targeting must be **LiDAR-driven**, and the arm must be commanded through your `ExecuteTrajectory` action client. Hard-coding positions or bypassing the action client → **0 points for Task 2**.

Same Control Architecture, New Mission

TASK 1 · asmr_arm_mission

~~push_block_mission · hardcoded Cartesian waypoints · arm only~~

TASK 2 · my_robot_mission

corridor_mission · LiDAR-reactive FSM · arm + locomotion

↓ cartesian target (x, z)

↓ velocity command (x, y, yaw)

ARM CONTROL — Task 1

- Inverse Kinematics
- ↓
- Interpolation
- ↓
- Trajectory Executor
- ↓
- PID Controller

LOCOMOTION CONTROL — MiniLab 1

- Velocity Smoother
- ↓
- Diff Drive Controller

↓ joint torque (τ)

↓ wheel torque (x, y, yaw)

Physics Engine

Mission Protocol

The arm can move. The robot can drive. **What decides what happens next?**

Spaghetti — one long callback or loop, `if/elif` chains, no explicit state. Works for simple scripts; collapses under complexity.

Finite State Machine — the robot is always in exactly one *state*. Each state defines its behaviour. *Transitions* fire when a condition is met and switch to the next state.

Behaviour Tree — modular, composable, restartable. More expressive than an FSM; overkill here.

An FSM is reactive by nature — transitions fire on sensor readings, not on a pre-computed plan.

Task 2 — Example of a Finite State Machine

Starting Point

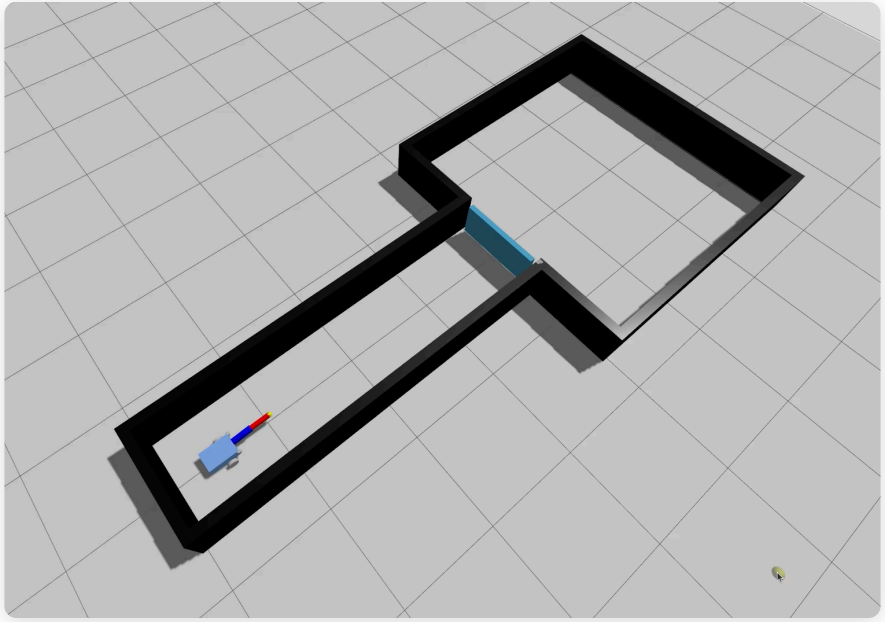
Package / File	Role	Origin
`arm.urdf.xacro`	mountable arm macro (brings its own `ros2_control`)	provided
`corridor_world.sdf`	straight corridor world with hinged door; poses in the header	provided
`my_robot_description`	**mount `arm.urdf.xacro` on your MiniLab 1 robot**	**you extend**
`my_robot_bringup`	**single-command bring-up of the whole stack**	**you build**
`my_robot_mission`	**LiDAR FSM — drives corridor, pushes door, passes through**	**you build**

Plus your full Task 1 stack — reused as-is. Launch: `ros2 launch my_robot_bringup corridor_world.launch.py` *· Run:* `ros2 run my_robot_mission corridor_mission`

My Approach (reference)

- **LiDAR sectors** — reuse `sector_min` (front / left / right) from MiniLab 1 to sense walls and the door.
- **A small state machine** — *Drive*, *Push*, *Pass*, *Done*: drive the corridor, stop at the door, push it open, advance.
- **Push = an action goal** — on *Push*, stop the base and send waypoints through the same `ExecuteTrajectory` client.
- **Retry, don't hang** — bounded retries; advance when the way clears.

> One approach — not the only one, and not prescribed. Pick what you can explain.



Reference solution: drive the corridor, push the door, pass through.